

Phase 4 — Docker Multi-Container Deployment

Harrington Capital plc | Infrastructure Engineer Simulation

Objective

Deploy a three-tier containerised application stack on HC-APP01 using Docker Compose. Nginx reverse proxy, Python Flask app, PostgreSQL database. All running, all networked, evidence committed to GitHub.

By the end of this phase you will have: - Three containers running: `hc-nginx`, `hc-app`, `hc-db` - Application reachable on `http://<HC-APP01-IP>:8080` - `docker ps` showing all containers `Up` - All compose files committed to `/docker` in your repo

BUILD

Step 1 — Ensure Docker is installed on HC-APP01

SSH into HC-APP01:

```
docker --version
docker compose version
```

If not installed:

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo usermod -aG docker $USER
newgrp docker
```

Step 2 — Clone the repo on HC-APP01

```
git clone https://github.com/YOUR-USERNAME/harrington-infrastructure-operations.git
cd harrington-infrastructure-operations/docker
```

Step 3 — Create the supporting application files

The `docker-compose.yml` references three directories that need content.

Create the Flask app:

```
mkdir -p app nginx db
```

Create `docker/app/app.py`:

```
from flask import Flask, jsonify
import os, psycopg2

app = Flask(__name__)

@app.route("/")
def index():
    return "<h1>Harrington Capital – Infrastructure Ops Portal</h1><p>Status: Running</p>"

@app.route("/health")
def health():
    return jsonify({"status": "healthy", "service": "hc-app", "environment": "prod"})

@app.route("/db-check")
def db_check():
    try:
        conn = psycopg2.connect(
            host=os.environ.get("DB_HOST", "db"),
            database=os.environ.get("DB_NAME", "harrington_ops"),
            user=os.environ.get("DB_USER", "hcapp"),
            password=os.environ.get("DB_PASSWORD", "HC@AppDB2026!")
        )
        conn.close()
        return jsonify({"status": "ok", "database": "connected"})
    except Exception as e:
        return jsonify({"status": "error", "detail": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create the Nginx config:

Create `docker/nginx/nginx.conf`:

```
events {
    worker_connections 1024;
}

http {
    upstream flask_app {
        server app:5000;
    }

    server {
        listen 80;
        server_name _;

        location / {
            proxy_pass http://flask_app;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /health {
            proxy_pass http://flask_app/health;
        }
    }
}
```

Create the DB init script:

Create `docker/db/init.sql`:

```
CREATE TABLE IF NOT EXISTS infrastructure_log (
    id          SERIAL PRIMARY KEY,
    timestamp   TIMESTAMPTZ DEFAULT NOW(),
    event_type  VARCHAR(50),
    host        VARCHAR(100),
    message     TEXT
);

INSERT INTO infrastructure_log (event_type, host, message)
VALUES ('STARTUP', 'hc-db', 'Database initialised for Harrington Capital ops portal');
```

These files are already in the repo — see `/docker/app/`, `/docker/nginx/`, `/docker/db/`.

Step 4 — Start the stack

```
cd harrington-infrastructure-operations/docker  
docker compose up -d
```

Wait approximately 60 seconds for the database health check to pass.

Step 5 — Verify all containers are running

```
docker ps
```

Must show three containers with status `Up` : - `hc-nginx` - `hc-app` - `hc-db`

Step 6 — Test the application

```
curl http://localhost:8080  
curl http://localhost:8080/health  
curl http://localhost:8080/db-check
```

Or open `http://<HC-APP01-IP>:8080` in your browser.

Step 7 — Commit

```
git add docker/  
git commit -m "feat(docker): multi-container stack deployed - Phase 4"  
git push origin main
```

VERIFY

Run each command and capture the output:

```
# Container status
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Container logs (no errors)
docker compose logs --tail=20

# Network inspection
docker network ls
docker network inspect hc-frontend

# Health endpoint
curl -s http://localhost:8080/health | python3 -m json.tool

# DB connectivity
curl -s http://localhost:8080/db-check | python3 -m json.tool

# Resource usage
docker stats --no-stream
```

SUBMIT

#	Evidence	How to get it
1	<code>docker ps</code> output	Copy full table output
2	Browser screenshot	<code>http://<HC-APP01-IP>:8080</code> showing the app
3	<code>/health</code> endpoint response	<code>curl</code> JSON output
4	<code>/db-check</code> endpoint response	<code>curl</code> JSON output — must show <code>"database": "connected"</code>
5	GitHub commit URL	Commit with all docker files

Anti-fake check: The `docker ps` output must show real container IDs (12-character hex strings) and `Up X minutes` status. These only exist if Docker is actually running.

Troubleshooting

Error	Fix
<code>hc-db is unhealthy</code>	Wait 30 more seconds — health check takes time
<code>hc-app</code> restarting	Check logs: <code>docker logs hc-app</code> — usually a DB connection issue
Port 8080 not reachable	Check UFW: <code>sudo ufw allow 8080</code>
<code>Cannot connect to Docker daemon</code>	Run <code>sudo systemctl start docker</code>
<code>docker compose: command not found</code>	Use <code>docker-compose</code> (with hyphen) for older installs